

MASTER'S THESIS DEFENSE

GPU-Initiated Communication and Application Scalability

From microbenchmarks to a conjugate gradient application

Franco Nicolas Merenda

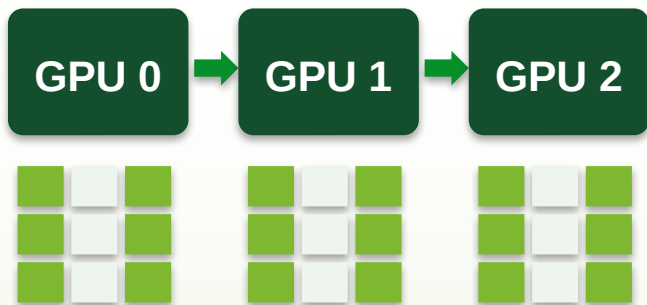
Università degli Studi di Salerno · Computer Science

Supervisor: Prof. Biagio Cosenza · Co-supervisor: Lorenzo Carpentieri

Academic year 2025–2026

Fast GPUs still wait for data

Scientific simulation

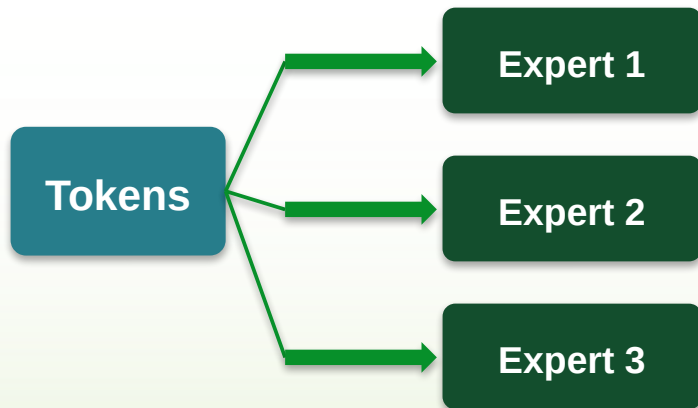


Exchange boundary values

Combine small global sums

Scalability depends on time spent waiting, not just GPU compute speed.

Distributed AI



Route activations across GPUs

Move the call into the kernel

GPU-initiated communication: GPU threads issue the communication operation.

Host-orchestrated path



Persistent GPU work



Why the interest now?

1

Smaller latency budgets

Inference repeatedly waits on small collectives.

2

Data-dependent traffic

MoE routing and irregular work need fine-grained control.

3

Better device interfaces

Symmetric memory and device APIs enable fused execution.

The opportunity: reduce orchestration and overlap useful work with communication.

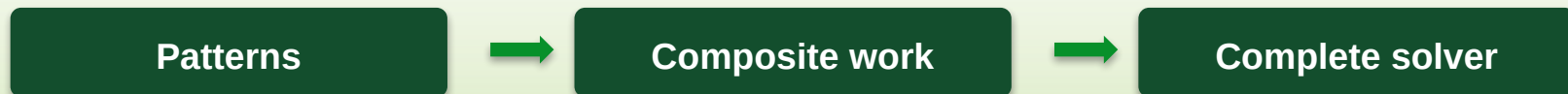
One ecosystem, different contracts

Path	Who issues the call?	What the programmer controls
GPU-aware MPI	CPU	Device buffers; message / collective semantics
Traditional NCCL	CPU, on a stream	Collectives and grouped peer transfers
NVSHMEM	CPU or GPU	Symmetric memory; puts, signals, ordering
NCCL device API / GIN	GPU kernel	Registered windows; device-side operations

OSHMPI: SHMEM over MPI oneCCL: collective interface SYCL: compute model

When does a faster path help?

- 1 Benefit** When does device initiation repay its costs?
- 2 Integration** What do memory and completion adaptations cost?
- 3 Transfer** Which benchmark observations survive in a solver?



What I built and evaluated

Unified benchmark

CUDA + SYCL
Seven configured stacks
Primitive + composite
patterns

Measured operation paths

oneCCL extensions

Grouped NCCL point-to-point
OSHMPI operation support
NVSHMEM runtime
foundation

NVSHMEM adapter: runtime validated; operations unsupported

Application extensions

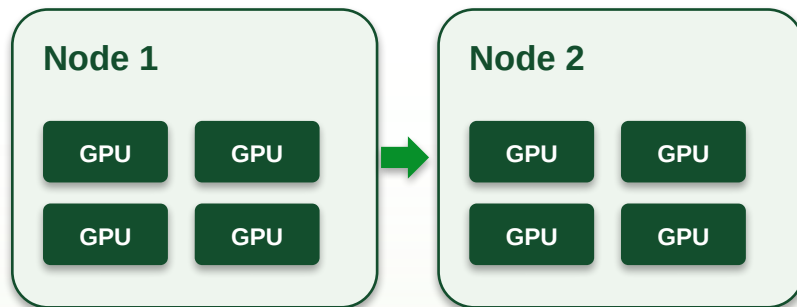
OSHMPI halo
communicator
SYCL implementation of
aCG
Complete-solver validation

Measure complete configured paths

Leonardo Booster

1–32 A100 GPUs · CUDA + SYCL
NVLink within nodes · InfiniBand across
nodes

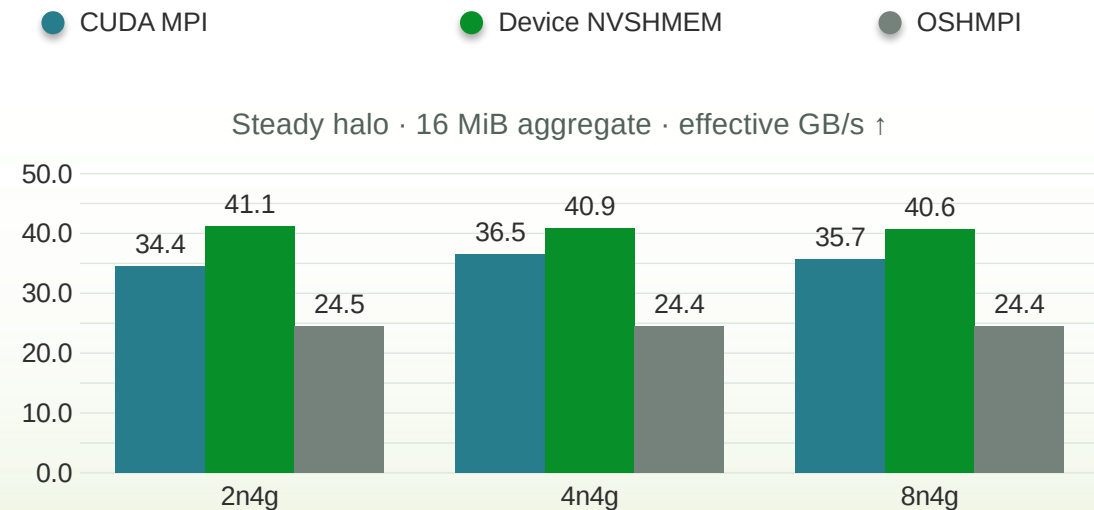
**Patterns → MoE / CG step → two
sparse-matrix solves**



Proxy-assisted NVSHMEM network posting

8n4g = 8 nodes × 4 GPUs per node. GPU count alone does not describe the path.

Persistence helps regular halos



40.6

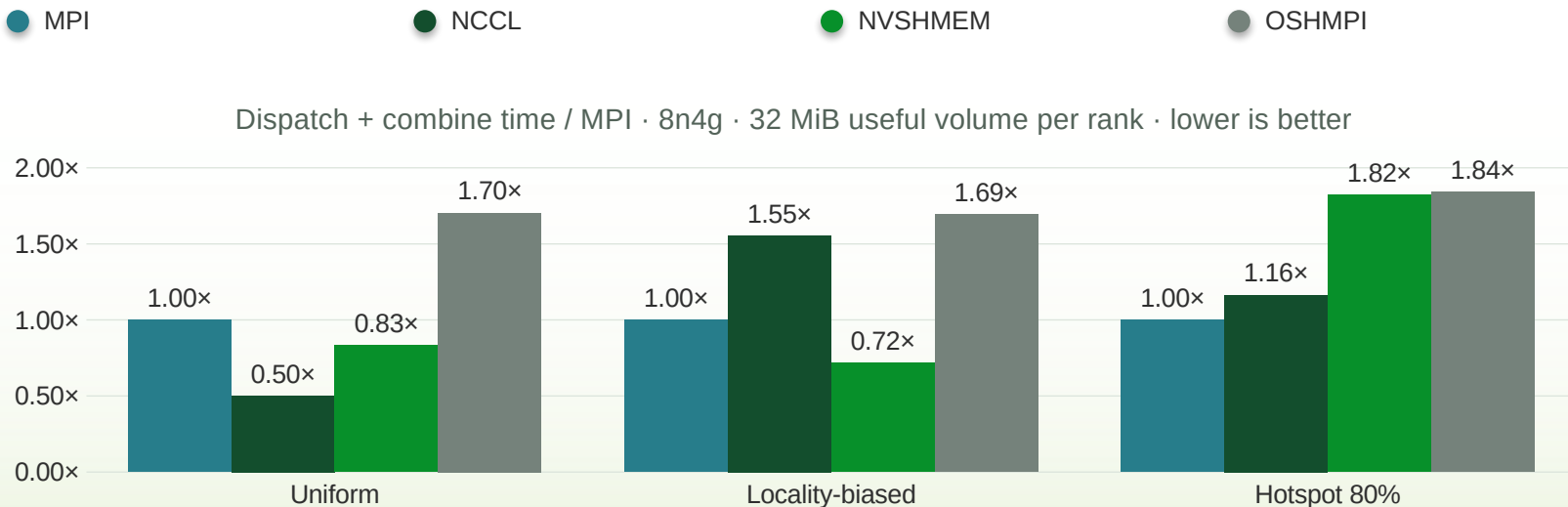
GB/s at 8n4g

MPI: 35.7 GB/s

100-exchange batches

The favorable regime is sustained neighbor traffic; small / isolated exchanges favor MPI.

Routing changes the winner



Same total volume and topology; a different payload distribution changes the ranking.

An adapter can change the path

oneCCL / NCCL

Grouped peer transfers
Asynchronous stream execution

0.98×

adapted / direct time
Bulk all-to-all · 8n4g

oneCCL / OSHMPI

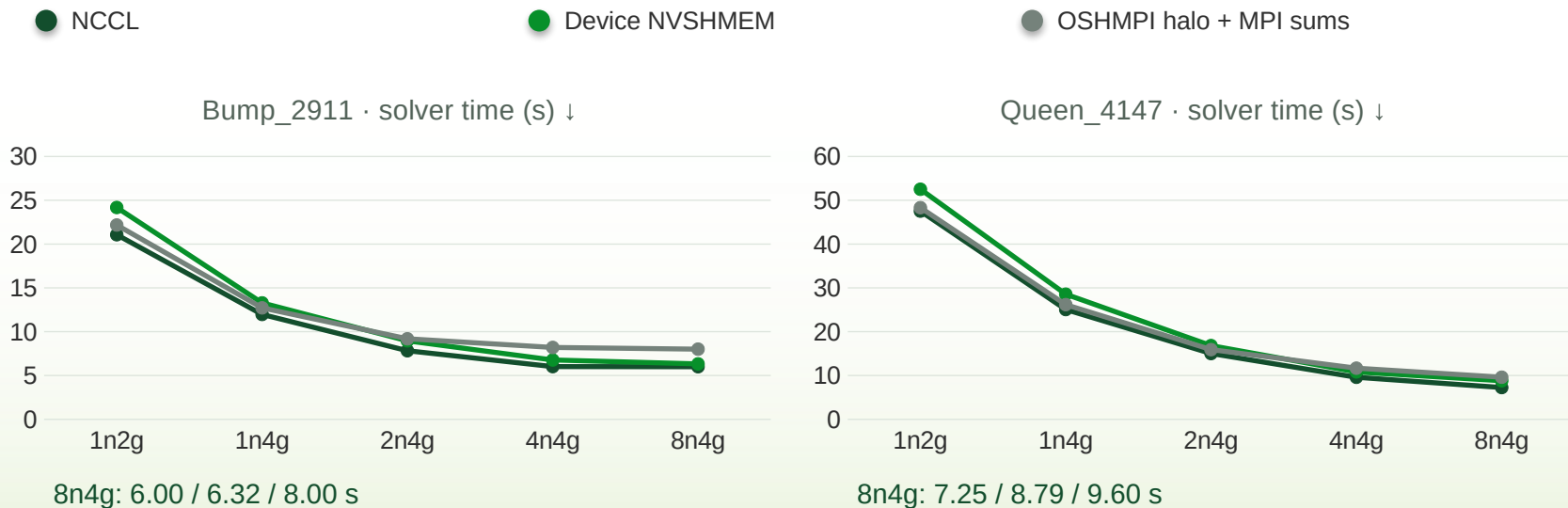
Symmetric staging buffers
Blocking completion adaptation

17.9×

adapted / direct time
16 MiB allreduce · 1n4g

The cost follows buffer movement and completion semantics, not the API name alone.

The solver is the deciding test



Device NVSHMEM gains on blocking OSHMPI at scale; no universal GPU-initiated lead.

What comes next?

1

NCCL device API + GIN



Compare verified proxy and GPU-posted paths

2

**Symmetric collectives:
within 7% of modeled bound**



Match initiation, memory, and completion

3

Portable device communication



Expose device handles through oneCCL / SYCL

Then replay real peer graphs and validate the gain in complete applications.

Choose for the application

1

Match the pattern

Payloads, peers, topology, and local work

2

Match completion

Where the result is ready and who can use it

3

Validate the solver

Measure the waiting that remains exposed

Thank you · Questions?

Full solver endpoints & protocol

8n4g · seconds · best / median residual-valid solver time

Backend	Bump_2911	Queen_4147
MPI	12.55 / 86.60	17.21 / 84.76
NCCL	6.00 / 6.37	7.25 / 7.51
Host NVSHMEM	13.48 / 13.73	14.68 / 14.88
OSHMPI halo + MPI	8.00 / 8.33	9.60 / 9.86
Device NVSHMEM	6.32 / 6.41	8.79 / 8.91

Solver: 3 allocations × 3 launches planned per configuration; completed valid runs retained.

Patterns: median of allocation means; 5 allocations, or 3 for all-to-all and CG step.

Sources: acg-summary.csv; thesis methodology. Best curves are representative minima, not typical-performance estimates.

SYCL can retain native performance

0.87–1.02×

Paired SYCL / native solver-time ratios
Within one node · both matrices

Keep the execution contract

Match layout and synchronization.
Across nodes, host completion can dominate.

This is performance retention on NVIDIA A100, not cross-vendor performance portability.

Transfer requires matched dependencies

Matched scalar reductions

MPI / NCCL, one FP64 sum

Mean relative differences:

7.8–11.1% across five topologies

Device CG-step scaling

Grid side 8192: 8.0% / 12.6%

Grid side 512: 582% / 853%

Bump / Queen mean deviations

Even an unchanged communication sequence can mislead when local work per GPU differs.

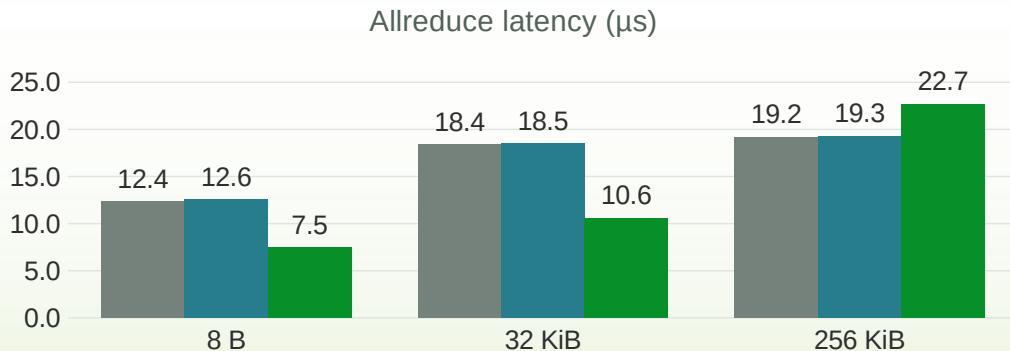
Newer NCCL: a follow-up candidate

1n4g · supplied three-run medians · allreduce latency in μs ↓

● 2.18.5 ordinary

● 2.31.2 ordinary

● 2.31.2 symmetric



Gain at small sizes
Regression at 256 KiB

Small all-to-all does not
share the small-size gain.

Symmetric allocation / registration does not establish device-side invocation.

What the evidence leaves open

Current limit	Experiment that addresses it
One cluster; proxy-assisted networking	Matched proxy / verified GPU-posted runs
Initiation changes with execution structure	Vary invocation, persistence, and batching separately
Regular halos; two solver matrices	Replay real peer graphs; expand application set
Older measured library paths	Compare newer NVSHMEM and NCCL device APIs
Portable adapter completion / staging	Device handles + stream-ordered completion

Report complete-path gains and losses; do not infer a universal library ordering.

Selected sources & artifacts

- 1 NVSHMEM mechanisms** Ma et al. (2026). Demystifying NVSHMEM. arXiv:2606.05951.
- 2 NCCL networking** Hamidouche et al. (2025). GPU-Initiated Networking for NCCL. arXiv:2511.15076.
- 3 Latency-bound collectives** Shen et al. (2026). Every Microsecond Matters. arXiv:2607.16100.
- 4 API definitions** NVIDIA. NCCL Device-Initiated Communication documentation. Checked 6 Oct 2026.
- 5 Measured evidence** Retained main-campaign tables, solver summaries, and thesis methodology.
- 6 Contributed software** Benchmark suite; three oneCCL extensions; OSHMPI and SYCL aCG extensions.